SOFTWARE REQUIREMENTS SPECIFICATION

For CampusFix: Campus Maintenance & Grievance Portal

Full Stack Application Development with ReactJS

ICT Academy of Kerala — Capstone Internship Project

Prepared by:

Megha K H

Department of Computer Science & Engineering

Sahrdaya College of Engineering and Technology

Date: 25.10.2026 | Version: 1.0

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive description of the functional, non-functional, and architectural requirements for **CampusFix**. CampusFix is an institutional web portal designed for educational institutions to streamline and digitize facility maintenance, grievance reporting, and technician resolution workflows.

1.2 Document Conventions

This document follows standard IEEE 830-1998 guidelines for software requirement specifications. Typographical conventions include boldface for critical system terms, monospaced font for code entities, endpoints, and database models.

1.3 Intended Audience

This document is intended for project evaluators at the Information and Communication Technology Academy of Kerala (ICTAK), software engineering mentors, institutional administrators, and future development contributors.

1.4 Project Scope

CampusFix replaces manual complaint registers and fragmented WhatsApp groups in educational institutions. The system provides a centralized digital portal where authenticated students and faculty can submit maintenance tickets (categorized by department, building, and urgency), monitor progress, and upvote existing tickets. It empowers administrative facility managers to triage tickets, assign technicians, update lifecycle statuses (*Pending* → *In Progress* → *Resolved*), and log official resolution remarks.

2. OVERALL DESCRIPTION

2.1 Product Perspective

CampusFix is an autonomous, cloud-enabled web application built on the three-tier MERN stack architecture (MongoDB, Express.js, React.js, and Node.js). It interfaces with a cloud-hosted MongoDB Atlas cluster for data persistence and enforces stateless authentication via JSON Web Tokens (JWT).

2.2 User Classes & Roles

User Role	Description & Responsibilities	Access Privileges
Student / Faculty (Reporter)	General campus members who identify infrastructure breakdowns or equipment faults.	Create tickets, view personal tickets, browse campus feed, upvote tickets, delete own pending tickets.
Facility Admin (Maintainer)	Campus maintenance staff, system administrators, or departmental facility managers.	View administrative metrics, update ticket status, append official resolution remarks, remove spam or duplicate tickets.

3. SPECIFIC SYSTEM FEATURES

3.1 Feature 1: User Authentication & Role Assignment

- **FR-1.1:** The system shall allow users to register with full name, unique college email, encrypted password, and department selection.
- **FR-1.2:** Passwords must be at least 6 characters in length and must be salted and hashed using `bcryptjs` (10 salt rounds) before database persistence.
- **FR-1.3:** The system shall issue a signed JSON Web Token (JWT) with a 30-day validity upon successful credential verification.
- **FR-1.4:** The system shall assign user roles (`student` or `admin`) during registration to dictate role-based access control (RBAC).

3.2 Feature 2: Maintenance Ticket Management

- **FR-2.1:** Authenticated students and faculty shall submit maintenance tickets specifying title, category (*IT & Lab, Electrical, Plumbing, Furniture, Cleanliness, Other*), campus location, urgency (*Low, Medium, High, Urgent*), and description.
- **FR-2.2:** Each ticket shall automatically capture the reporter's ID, submission timestamp, default `Pending` status, and an empty upvotes array.
- **FR-2.3:** Users shall view a dedicated "My Tickets" dashboard showing only tickets authored by their authenticated account.
- **FR-2.4:** Authors may delete their own tickets provided the ticket remains in the `Pending` state.

3.3 Feature 3: Campus Feed, Search & Dynamic Filtering

- **FR-3.1:** The application shall display a centralized feed displaying all active campus maintenance tickets to prevent redundant reports.
- **FR-3.2:** The feed shall support live multi-parameter filtering by Category and Status.
- **FR-3.3:** The search feature shall perform real-time substring matching across ticket titles, descriptions, and campus locations.
- **FR-3.4:** Real-time summary metric cards shall dynamically display counts for Total Filtered, Pending, In Progress, and Resolved tickets.

3.4 Feature 4: Administrative Resolution & Facility Remarks

- **FR-4.1:** Facility Admins shall possess exclusive access to the "Manage Status" drawer on ticket cards.
- **FR-4.2:** Admins can update ticket status between `Pending`, `In Progress`, and `Resolved`.
- **FR-4.3:** Admins can append official maintenance updates and technician notes (e.g., "*Plumber assigned; pipeline cleared*").
- **FR-4.4:** Status updates shall immediately recalculate campus metric counters across the user interface.

3.5 Feature 5: Peer Upvoting

- **FR-5.1:** Any authenticated campus member can upvote a ticket to indicate shared urgency.
- **FR-5.2:** The upvote action is atomic: clicking once increments the counter; clicking again retracts the upvote.

4. EXTERNAL INTERFACE REQUIREMENTS

4.1 User Interfaces

The client interface is implemented in React 18 utilizing modern CSS flexbox and grid layouts. The interface strictly adheres to Section 7.3 of the ICTAK guidelines, ensuring every data-driven view provides four distinct feedback states:

- **Loading State:** Visual loader spinners during asynchronous fetch calls.
- **Success State:** Formatted cards with color-coded status badges (Amber for *Pending*, Blue for *In Progress*, Green for *Resolved*).
- **Empty State:** Intuitive placeholder graphics with an actionable *"Report New Issue"* button when no matching records exist.
- **Error State:** Non-technical error banners with retry triggers.

4.2 Software Interfaces & REST API Endpoints

Method	Endpoint Path	Access Tier	Description
POST	/api/auth/register	Public	Registers student or admin account
POST	/api/auth/login	Public	Authenticates credentials and returns JWT token
GET	/api/auth/me	Private	Fetches authenticated user profile
GET	/api/issues	Private	Queries all tickets with search and category filters
POST	/api/issues	Private	Creates new maintenance ticket
GET	/api/issues/my	Private	Retrieves tickets created by logged-in user
PUT	/api/issues/:id/status	Admin Only	Updates lifecycle status and resolution remarks
PUT	/api/issues/:id/upvote	Private	Toggles peer upvote on a ticket
DELETE	/api/issues/:id	Owner / Admin	Deletes specified ticket

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Security Requirements

- **Password Hashing:** Plaintext passwords are never logged or stored; passwords undergo salted hashing via `bcryptjs`.
- **Secret Isolation:** JWT secrets, database connection URIs, and port numbers are strictly loaded through server-side `.env` files excluded from source control.
- **Role Enforcement:** API endpoints verify user authorization claims in JWT payloads to block privilege escalation.

5.2 Performance & Reliability

- Sub-second API response times through Mongoose indexed queries.
- Vite-powered single-page application bundling delivering instant client navigation without full-page reloads.
- Automatic reconnect logic configured on the MongoDB Atlas database cluster.